



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Информатика, искусственный интеллект и системы управления

КАФЕДРА Системы обработки информации и управления

ОТЧЕТ

по Лабораторной работе № 4.2

по дисциплине «**МЕТОДЫ МАШИННОГО ОБУЧЕНИЯ**»

Тема: «**Необходимые и достаточные условия существования условного экстремума**»

Студент	<u>ИУ5И-22М</u>	<u></u>	<u>Тавуз Мохамад</u>
	(группа)	(подпись, дата)	(И.О. Фамилия)
Преподаватель		<u></u>	<u>Ю.Е.Гапанюк</u>
		(подпись, дата)	(И.О. Фамилия)

2026 г.

1) Введение

Методы условной оптимизации являются важным разделом математического программирования, так как во многих практических задачах требуется найти экстремум целевой функции при наличии ограничений. В отличие от задач безусловной оптимизации, решение должно не только обеспечивать минимальное или максимальное значение функции, но и принадлежать области допустимых решений.

Цель данной лабораторной работы — исследование необходимых и достаточных условий существования условного экстремума функции с учетом ограничений, а также вычисление экстремума заданной функции для выбранного варианта. В ходе работы рассматривается вариант 2, для которого необходимо записать условия Куна–Таккера, найти стационарные точки, проверить их допустимость, исследовать условия второго порядка и подтвердить найденное решение численным методом.

Для выполнения работы использовалась среда Google Colab и язык программирования Python. В программе были применены библиотеки `numpy`, `pandas`, `matplotlib`, `sympy` и `scipy.optimize`, которые использовались для символьных вычислений, численного решения задачи, проверки ограничений и построения графиков.

2) Описание задания

В соответствии с заданием лабораторной работы необходимо найти экстремум функции при наличии ограничений и исследовать найденные точки с помощью необходимых и достаточных условий условного экстремума.

В работе рассматривается вариант 2. Для данного варианта задана целевая функция:

$$f(x) = x_1^2 + (x_2 - x_1)^3 \rightarrow \text{extr}$$

при ограничениях:

$$g_1(x) = x_1^2 + x_2^2 - 1 \leq 0,$$

$$g_2(x) = x_1 - x_2 \leq 0.$$

Таким образом, область допустимых решений определяется пересечением круга единичного радиуса с полуплоскостью $x_1 \leq x_2$. В ходе работы необходимо определить допустимые стационарные точки, проверить выполнение условий Куна–Таккера, исследовать условия второго порядка и подтвердить результат численным методом условной оптимизации.

3) Текст программы и экранные формы с примерами выполнения

В данном разделе приведены основные этапы программной реализации решения задачи условной оптимизации. Все вычисления были выполнены в среде Google Colab с использованием языка Python.

На первом этапе были подключены библиотеки, необходимые для выполнения лабораторной работы. Библиотека `numpy` использовалась для численных вычислений и работы с массивами, `pandas` — для формирования таблиц результатов, `matplotlib` — для построения графиков. Библиотека `sympy` применялась для символьного решения условий Куна–Таккера, а функция `minimize` из `scipy.optimize` использовалась для численной проверки найденного решения методом SLSQP.

 # Импорт необходимых библиотек для выполнения лабораторной работы

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sympy as sp
import time

from scipy.optimize import minimize

print("Libraries imported successfully")
```

... Libraries imported successfully

Рисунок 1 — Импорт необходимых библиотек

3.1. Задание варианта, целевой функции и ограничений задачи

На следующем этапе был задан номер варианта и определены основные функции задачи. Для варианта 2 была реализована целевая функция $f(x) = x_1^2 + (x_2 - x_1)^3$. Также были заданы две функции ограничений: $g_1(x) = x_1^2 + x_2^2 - 1 \leq 0$ и $g_2(x) = x_1 - x_2 \leq 0$.

Дополнительно были реализованы вспомогательные функции для вычисления суммарного нарушения ограничений и проверки допустимости точки. Эти функции используются далее при анализе тестовых точек, фильтрации найденных решений и итоговой проверке результата.

Задание варианта, целевой функции и ограничений задачи

```
variant = 2

def objective_function(x):
    x1, x2 = x
    return x1**2 + (x2 - x1)**3

def constraints(x):
    x1, x2 = x
    g1 = x1**2 + x2**2 - 1
    g2 = x1 - x2
    return np.array([g1, g2], dtype=float)

def constraint_violation(x):
    g = constraints(x)
    return np.sum(np.maximum(g, 0.0) ** 2)

def is_feasible(x, tolerance=1e-8):
    return np.all(constraints(x) <= tolerance)

print("Variant:", variant)
print("Objective function: f(x) = x1^2 + (x2 - x1)^3")
print("Constraints:")
print("g1(x) = x1^2 + x2^2 - 1 <= 0")
print("g2(x) = x1 - x2 <= 0")
```

```
... Variant: 2
Objective function: f(x) = x1^2 + (x2 - x1)^3
Constraints:
g1(x) = x1^2 + x2^2 - 1 <= 0
g2(x) = x1 - x2 <= 0
```

Рисунок 2 — Задание варианта, целевой функции и ограничений

3.2. Проверка целевой функции, ограничений и допустимости тестовых точек

После задания целевой функции и ограничений была выполнена проверка корректности их программной реализации на нескольких тестовых точках. Для каждой точки вычислялось значение целевой функции $f(x)$, значения ограничений $g_1(x)$ и $g_2(x)$, величина нарушения ограничений, а также признак допустимости точки.

Результаты проверки показали, что точки $(0, 0)$, $(0.5, 0.5)$ и $(-0.5, 0.5)$ принадлежат допустимой области, так как для них выполняются оба ограничения. Точка $(1, 1)$ не является допустимой, поскольку нарушает ограничение $g_1(x) = x_1^2 + x_2^2 - 1 \leq 0$. Точка $(0.7, 0.8)$ также оказалась недопустимой из-за нарушения первого ограничения.

Проверка целевой функции, ограничений и допустимости тестовых точек

```
test_points = [  
    np.array([0.0, 0.0]),  
    np.array([0.5, 0.5]),  
    np.array([1.0, 1.0]),  
    np.array([-0.5, 0.5]),  
    np.array([0.7, 0.8])  
]  
  
for point in test_points:  
    print("Point:", point)  
    print("f(x):", objective_function(point))  
    print("g(x):", constraints(point))  
    print("Constraint violation:", constraint_violation(point))  
    print("Feasible:", is_feasible(point))  
    print("-" * 50)
```

Рисунок 3 — Код проверки целевой функции, ограничений и допустимости тестовых точек

```

... Point: [0. 0.]
    f(x): 0.0
    g(x): [-1. 0.]
    Constraint violation: 0.0
    Feasible: True
-----
    Point: [0.5 0.5]
    f(x): 0.25
    g(x): [-0.5 0. ]
    Constraint violation: 0.0
    Feasible: True
-----
    Point: [1. 1.]
    f(x): 1.0
    g(x): [1. 0.]
    Constraint violation: 1.0
    Feasible: False
-----
    Point: [-0.5 0.5]
    f(x): 1.25
    g(x): [-0.5 -1. ]
    Constraint violation: 0.0
    Feasible: True
-----
    Point: [0.7 0.8]
    f(x): 0.49099999999999994
    g(x): [ 0.13 -0.1 ]
    Constraint violation: 0.016900000000000003
    Feasible: False
-----

```

Рисунок 4 — Результат проверки целевой функции, ограничений и допустимости тестовых точек

3.3. Визуализация области допустимых решений

Для наглядного представления задачи была построена область допустимых решений и линии уровня целевой функции. Область допустимых решений определяется двумя ограничениями: $g_1(x) = x_1^2 + x_2^2 - 1 \leq 0$ задает круг единичного радиуса, а $g_2(x) = x_1 - x_2 \leq 0$ задает полуплоскость $x_1 \leq x_2$.

На графике допустимая область показана как пересечение этих двух множеств. Также были построены линии уровня целевой функции $f(x) = x_1^2 + (x_2 - x_1)^3$. Такая визуализация позволяет предварительно оценить расположение возможного экстремума и понять, какие ограничения могут быть активными в найденной точке.

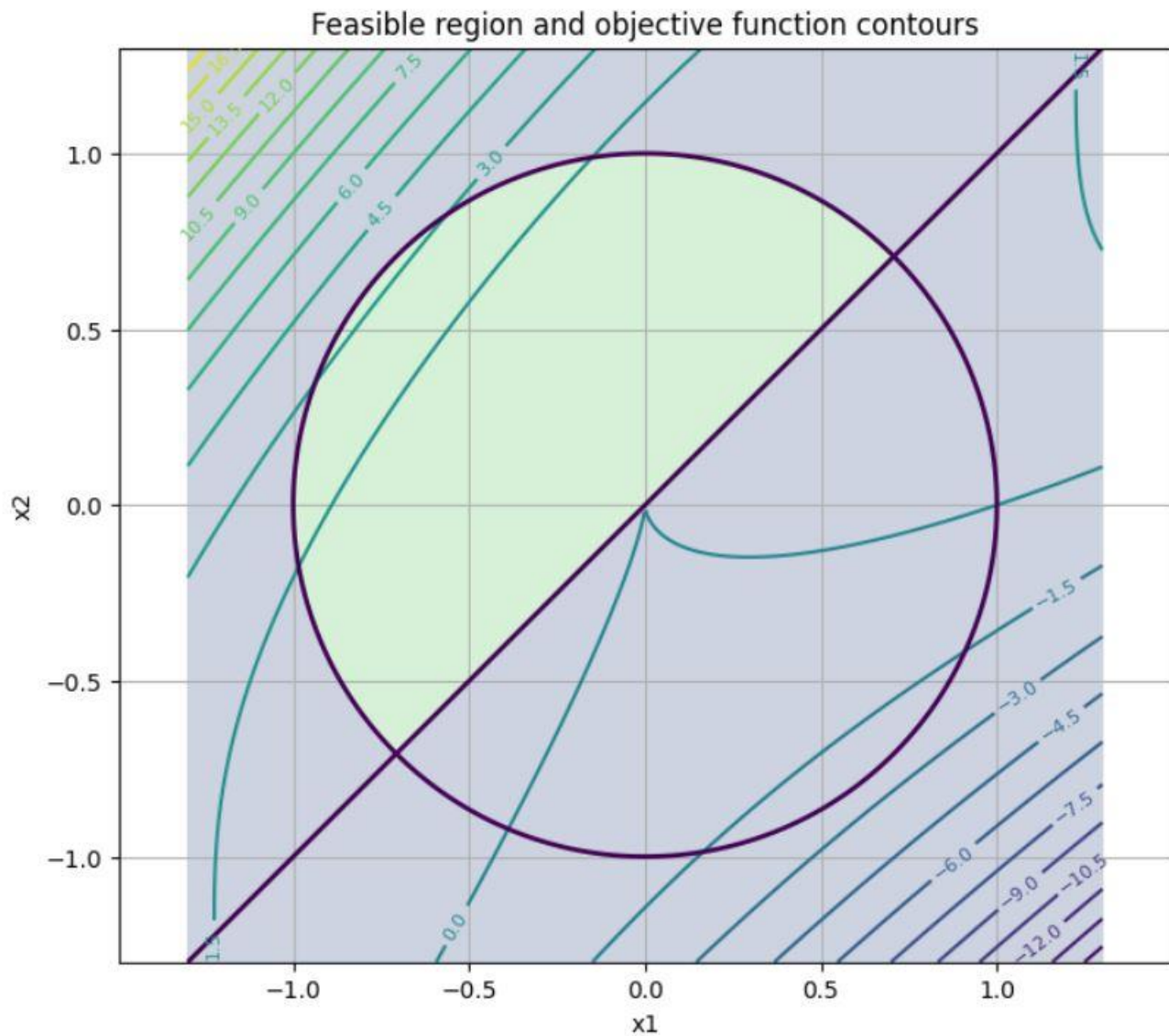


Рисунок 5 — Область допустимых решений и линии уровня целевой функции

3.4. Символьное решение условий Куна–Таккера

На данном этапе была составлена классическая функция Лагранжа для задачи с ограничениями-неравенствами:

$$L(x, \mu) = f(x) + \mu_1 g_1(x) + \mu_2 g_2(x),$$

где μ_1 и μ_2 — множители Лагранжа, соответствующие ограничениям $g_1(x)$ и $g_2(x)$. Для поиска условно-стационарных точек были записаны условия стационарности по переменным x_1 и x_2 , условия допустимости, условия неотрицательности множителей и условия дополняющей нежесткости.

Так как заранее неизвестно, какие ограничения являются активными в точке экстремума, были рассмотрены четыре возможных случая: отсутствие активных ограничений, активность только ограничения g_1 , активность только ограничения g_2 , а также одновременная активность обоих ограничений. Для каждого случая система уравнений была решена символично с помощью библиотеки `sympy`.

▶ # Символьное решение условий Куна-Таккера для разных активных ограничений

```
x1, x2, mu1, mu2 = sp.symbols("x1 x2 mu1 mu2", real=True)
```

```
f_sym = x1**2 + (x2 - x1)**3
```

```
g1_sym = x1**2 + x2**2 - 1
```

```
g2_sym = x1 - x2
```

```
L_sym = f_sym + mu1 * g1_sym + mu2 * g2_sym
```

```
stationarity_equations = [  
    sp.diff(L_sym, x1),  
    sp.diff(L_sym, x2)  
]
```

```
cases = {  
    "No active constraints": [  
        stationarity_equations[0],  
        stationarity_equations[1],  
        mu1,  
        mu2  
    ],  
    "Only g1 active": [  
        stationarity_equations[0],  
        stationarity_equations[1],  
        g1_sym,  
        mu2  
    ],  
    "Only g2 active": [  
        stationarity_equations[0],  
        stationarity_equations[1],  
        mu1,  
        g2_sym  
    ],  
    "g1 and g2 active": [  
        stationarity_equations[0],  
        stationarity_equations[1],  
        g1_sym,  
        g2_sym  
    ]  
}
```

```
solutions = []
```

```
for case name, equations in cases.items():
```



```

for case_name, equations in cases.items():
    print("=" * 70)
    print("Case:", case_name)

    case_solutions = sp.solve(equations, [x1, x2, mu1, mu2], dict=True)

    for sol in case_solutions:
        x1_complex = complex(sp.N(sol[x1]))
        x2_complex = complex(sp.N(sol[x2]))
        mu1_complex = complex(sp.N(sol.get(mu1, 0)))
        mu2_complex = complex(sp.N(sol.get(mu2, 0)))

        if (
            abs(x1_complex.imag) > 1e-8 or
            abs(x2_complex.imag) > 1e-8 or
            abs(mu1_complex.imag) > 1e-8 or
            abs(mu2_complex.imag) > 1e-8
        ):
            continue

        x_candidate = np.array([
            x1_complex.real,
            x2_complex.real
        ])

        mu1_value = mu1_complex.real
        mu2_value = mu2_complex.real

        g_values = constraints(x_candidate)
        f_value = objective_function(x_candidate)
        feasible = is_feasible(x_candidate)
        kkt_dual_feasible = (mu1_value >= -1e-8) and (mu2_value >= -1e-8)

        result = {
            "case": case_name,
            "x1": x_candidate[0],
            "x2": x_candidate[1],
            "mu1": mu1_value,
            "mu2": mu2_value,
            "f": f_value,
            "g1": g_values[0],
            "g2": g_values[1],
            "feasible": feasible,
            "dual_feasible": kkt_dual_feasible
        }

        solutions.append(result)
        print(result)

solutions_df = pd.DataFrame(solutions)
solutions_df

```

Рисунок 6 — Код символьного решения условий Куна-Таккера

	case	x1	x2	mu1	mu2	f	g1	g2	feasible	dual_feasible
0	No active constraints	0.000000	0.000000	0.000000	0.000000	0.000000	-1.000000e+00	0.000000	True	True
1	Only g1 active	-0.784506	0.620122	-4.772397	0.000000	3.386748	0.000000e+00	-1.404627	True	False
2	Only g1 active	0.620122	-0.784506	3.772397	0.000000	-2.386748	0.000000e+00	1.404627	False	True
3	Only g1 active	0.894040	0.447988	-0.666186	0.000000	0.710559	0.000000e+00	0.446052	False	False
4	Only g1 active	0.447988	0.894040	-0.333814	0.000000	0.289441	0.000000e+00	-0.446052	True	False
5	Only g2 active	0.000000	0.000000	0.000000	0.000000	0.000000	-1.000000e+00	0.000000	True	True
6	g1 and g2 active	0.707107	0.707107	-0.500000	-0.707107	0.500000	2.220446e-16	0.000000	True	False
7	g1 and g2 active	-0.707107	-0.707107	-0.500000	0.707107	0.500000	2.220446e-16	0.000000	True	False

Рисунок 7 — Результаты символьного решения условий Куна-Таккера

3.5. Фильтрация допустимых точек Куна–Таккера

После символьного решения систем уравнений были отобраны только те точки, которые удовлетворяют условиям допустимости прямой задачи и условиям допустимости двойственной задачи. Для этого из общего набора найденных решений были оставлены точки, для которых выполняются ограничения $g_1(x) \leq 0$, $g_2(x) \leq 0$, а также условия $\mu_1 \geq 0$ и $\mu_2 \geq 0$.

В результате фильтрации была получена точка $x^* = (0, 0)$. Для данной точки значение целевой функции равно $f(x^*) = 0$. Значения ограничений составляют $g_1(x^*) = -1$ и $g_2(x^*) = 0$, поэтому точка принадлежит допустимой области. При этом ограничение g_2 является активным, так как его значение равно нулю, а ограничение g_1 является пассивным.

▶ # Фильтрация допустимых точек, удовлетворяющих условиям Куна-Таккера

```
valid_kkt_points = solutions_df[
    (solutions_df["feasible"] == True) &
    (solutions_df["dual_feasible"] == True)
].copy()

valid_kkt_points = valid_kkt_points.drop_duplicates(
    subset=["x1", "x2", "f"],
    keep="first"
).reset_index(drop=True)

print("Valid KKT points:")
display(valid_kkt_points)

best_point_index = valid_kkt_points["f"].idxmin()
best_point = valid_kkt_points.loc[best_point_index]

print("Best point among valid KKT candidates:")
print("x* =", np.array([best_point["x1"], best_point["x2"]]))
print("f(x*) =", best_point["f"])
print("g(x*) =", np.array([best_point["g1"], best_point["g2"]]))
```

Рисунок 8 — Код фильтрации допустимых точек Куна-Таккера

... Valid KKT points:

	case	x1	x2	mu1	mu2	f	g1	g2	feasible	dual_feasible
0	No active constraints	0.0	0.0	0.0	0.0	0.0	-1.0	0.0	True	True

Best point among valid KKT candidates:
x* = [0. 0.]
f(x*) = 0.0
g(x*) = [-1. 0.]

Рисунок 9 — Результат фильтрации допустимых точек Куна-Таккера

3.6. Определение условного максимума

Так как в задании требуется найти экстремум функции, дополнительно был выполнен поиск условного максимума среди найденных точек Куна-Таккера. Для условного максимума в задаче с ограничениями вида $g_i(x) \leq 0$ были отобраны допустимые точки, для которых множители Лагранжа имеют неположительные значения.

В результате фильтрации были рассмотрены допустимые кандидаты на условный максимум. Наибольшее значение целевой функции было получено в точке $x_{\max} \approx (-0.784506, 0.620122)$. Для данной точки значение целевой функции составляет $f(x_{\max}) \approx 3.386748$. Значения ограничений равны $g_1(x_{\max}) = 0$ и $g_2(x_{\max}) \approx -1.404627$, поэтому точка принадлежит допустимой области, а активным является ограничение g_1 .

Таким образом, для выбранного варианта были найдены как условный минимум $x_{\min} = (0, 0)$, так и условный максимум $x_{\max} \approx (-0.784506, 0.620122)$.

```
▶ # Определение условного максимума среди точек Куна-Таккера

valid_kkt_max_points = solutions_df[
    (solutions_df["feasible"] == True) &
    (solutions_df["mu1"] <= 1e-8) &
    (solutions_df["mu2"] <= 1e-8)
].copy()

valid_kkt_max_points = valid_kkt_max_points.drop_duplicates(
    subset=["x1", "x2", "f"],
    keep="first"
).reset_index(drop=True)

print("Valid KKT points for maximum:")
display(valid_kkt_max_points)

max_point_index = valid_kkt_max_points["f"].idxmax()
max_point = valid_kkt_max_points.loc[max_point_index]

x_max = np.array([max_point["x1"], max_point["x2"]])

print("Best point for maximum among valid KKT candidates:")
print("x_max =", x_max)
print("f(x_max) =", max_point["f"])
print("g(x_max) =", np.array([max_point["g1"], max_point["g2"]]))
print("mu_max =", np.array([max_point["mu1"], max_point["mu2"]]))
```

Рисунок 10 — Код определения условного максимума

```

... Valid KKT points for maximum:

```

	case	x1	x2	mu1	mu2	f	g1	g2	feasible	dual_feasible
0	No active constraints	0.000000	0.000000	0.000000	0.000000	0.000000	-1.000000e+00	0.000000	True	True
1	Only g1 active	-0.784506	0.620122	-4.772397	0.000000	3.386748	0.000000e+00	-1.404627	True	False
2	Only g1 active	0.447988	0.894040	-0.333814	0.000000	0.289441	0.000000e+00	-0.446052	True	False
3	g1 and g2 active	0.707107	0.707107	-0.500000	-0.707107	0.500000	2.220446e-16	0.000000	True	False

```

Best point for maximum among valid KKT candidates:
x_max = [-0.78450565  0.62012167]
f(x_max) = 3.3867477840250686
g(x_max) = [ 0.          -1.40462732]
mu_max = [-4.77239712  0.          ]

```

Рисунок 11 — Результат определения условного максимума

3.7. Проверка условий второго порядка

Для найденной точки $x^* = (0, 0)$ была выполнена проверка условий второго порядка. Для этого была построена матрица Гессе функции Лагранжа по переменным x_1 и x_2 , после чего она была вычислена в найденной точке с учетом соответствующих множителей Лагранжа.

В результате была получена матрица Гессе:

$$H = \begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$$

Собственные значения данной матрицы равны 2 и 0. Следовательно, матрица является положительно полуопределенной. Это означает, что необходимое условие второго порядка для локального минимума в найденной точке выполняется.



Проверка условий второго порядка в найденной точке

```
hessian_L = sp.hessian(L_sym, (x1, x2))

x_star = np.array([best_point["x1"], best_point["x2"]])
mu_star = np.array([best_point["mu1"], best_point["mu2"]])

hessian_at_star = hessian_L.subs({
    x1: x_star[0],
    x2: x_star[1],
    mu1: mu_star[0],
    mu2: mu_star[1]
})

hessian_at_star_np = np.array(hessian_at_star, dtype=float)
eigenvalues = np.linalg.eigvals(hessian_at_star_np)

print("x* =", x_star)
print("mu* =", mu_star)
print("Hessian of Lagrangian at x*:")
print(hessian_at_star_np)
print("Eigenvalues:", eigenvalues)

print("\nSecond-order condition analysis:")
if np.all(eigenvalues >= -1e-8):
    print("The Hessian is positive semidefinite.")
    print("The second-order necessary condition for a local minimum is satisfied.")
else:
    print("The Hessian is not positive semidefinite.")
    print("The second-order necessary condition is not satisfied.")
```

Рисунок 12 — Код проверки условий второго порядка

```
... x* = [0. 0.]
    mu* = [0. 0.]
    Hessian of Lagrangian at x*:
    [[2. 0.]
     [0. 0.]]
    Eigenvalues: [2. 0.]

    Second-order condition analysis:
    The Hessian is positive semidefinite.
    The second-order necessary condition for a local minimum is satisfied.
```

Рисунок 13 — Результат проверки условий второго порядка

3.8. Численная проверка решения методом SLSQP

Для подтверждения результата, полученного с помощью условий Куна–Таккера, была выполнена численная проверка решения задачи условной оптимизации. Для этого использовалась функция `minimize` из библиотеки `scipy.optimize` с методом SLSQP, который позволяет решать задачи нелинейной оптимизации с ограничениями.

В качестве начальной точки была выбрана точка $x_0 = (0.2, 0.2)$, принадлежащая допустимой области. Ограничения были записаны в форме, используемой методом SLSQP: $1 - x_1^2 - x_2^2 \geq 0$ и $x_2 - x_1 \geq 0$. В результате численного решения была найдена точка, близкая к $x^* = (0, 0)$, а значение целевой функции оказалось близким к нулю. Это подтверждает корректность аналитического решения.

```
# Численное решение задачи условной оптимизации методом SLSQP

x0 = np.array([0.2, 0.2])

scipy_constraints = [
    {
        "type": "ineq",
        "fun": lambda x: 1 - x[0]**2 - x[1]**2
    },
    {
        "type": "ineq",
        "fun": lambda x: x[1] - x[0]
    }
]

start_time = time.time()

numerical_result = minimize(
    objective_function,
    x0,
    method="SLSQP",
    constraints=scipy_constraints,
    options={
        "ftol": 1e-12,
        "maxiter": 1000,
        "disp": False
    }
)

execution_time = time.time() - start_time

x_numerical = numerical_result.x
f_numerical = objective_function(x_numerical)
g_numerical = constraints(x_numerical)

print("Numerical optimization result:")
print("Success:", numerical_result.success)
print("Message:", numerical_result.message)
print("Found point:", x_numerical)
print("Function value:", f_numerical)
print("Constraints:", g_numerical)
print("Constraint violation:", constraint_violation(x_numerical))
print("Feasible:", is_feasible(x_numerical))
print("Iterations:", numerical_result.nit)
print("Execution time:", execution_time)
```

Рисунок 14 — Код численного решения задачи условной оптимизации методом SLSQP


```
... Numerical optimization result:
Success: True
Message: Optimization terminated successfully
Found point: [-7.4538278e-09  7.2792057e-05]
Function value: 3.858761336636238e-13
Constraints: [-9.99999995e-01 -7.27995108e-05]
Constraint violation: 0.0
Feasible: True
Iterations: 18
Execution time: 0.030282020568847656
```

Рисунок 15 — Результат численного решения задачи условной оптимизации методом SLSQP

3.9. Сравнение аналитического и численного решений

После получения аналитического решения с помощью условий Куна–Таккера и численного решения методом SLSQP было выполнено их сравнение. Для этого была сформирована таблица, содержащая координаты найденных точек, значение целевой функции, значения ограничений и признак допустимости решения.

Результаты сравнения показали, что аналитическое решение и численное решение практически совпадают. В обоих случаях была получена точка, близкая к $x^* = (0, 0)$, значение целевой функции равно нулю или близко к нулю, а ограничения выполняются. Это подтверждает правильность найденного условного экстремума.

Сравнение аналитического и численного решений задачи

```
comparison_data = [  
    {  
        "Method": "KKT symbolic solution",  
        "x1": x_star[0],  
        "x2": x_star[1],  
        "f(x)": objective_function(x_star),  
        "g1(x)": constraints(x_star)[0],  
        "g2(x)": constraints(x_star)[1],  
        "Feasible": is_feasible(x_star)  
    },  
    {  
        "Method": "SLSQP numerical solution",  
        "x1": x_numerical[0],  
        "x2": x_numerical[1],  
        "f(x)": f_numerical,  
        "g1(x)": g_numerical[0],  
        "g2(x)": g_numerical[1],  
        "Feasible": is_feasible(x_numerical)  
    }  
]  
  
comparison_df = pd.DataFrame(comparison_data)  
comparison_df
```

Рисунок 16 — Код сравнения аналитического и численного решений

...	Method	x1	x2	f(x)	g1(x)	g2(x)	Feasible
0	KKT symbolic solution	0.000000e+00	0.000000	0.000000e+00	-1.0	0.000000	True
1	SLSQP numerical solution	-7.453828e-09	0.000073	3.858761e-13	-1.0	-0.000073	True

Рисунок 17 — Таблица сравнения аналитического и численного решений

3.10. Визуализация найденного решения

Для наглядного анализа найденное решение было нанесено на график области допустимых решений. На графике отображены линии уровня целевой функции, границы ограничений и найденные точки, полученные двумя способами: с помощью условий Куна–Таккера и методом SLSQP.

Обе точки расположены в одной области и практически совпадают с точкой $x^* = (0, 0)$. Это показывает, что аналитическое и численное решения согласуются между собой. Кроме того, график подтверждает, что найденная точка принадлежит допустимой области, так как она удовлетворяет ограничениям задачи.

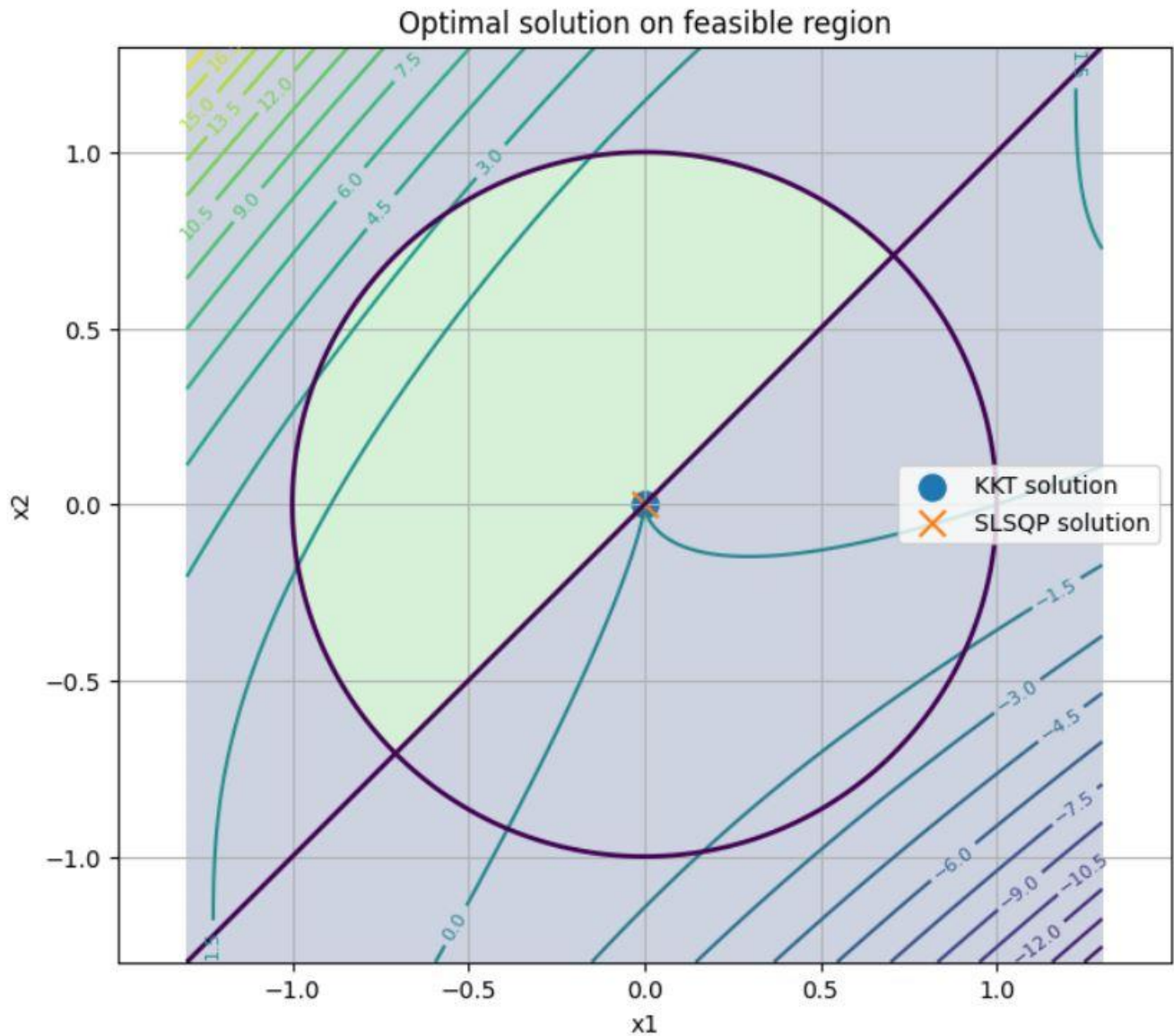


Рисунок 18 — Найденное решение на области допустимых решений

3.11. Итоговая проверка найденного решения

На заключительном этапе была выполнена итоговая проверка найденного решения. Для точки $x^* = (0, 0)$ были повторно вычислены значение целевой функции, значения ограничений, величина нарушения ограничений, признак допустимости, градиент целевой функции и норма градиента.

Результаты проверки показали, что $f(x^*) = 0$, ограничения имеют значения $g(x^*) = (-1, 0)$, а величина нарушения ограничений равна 0. Следовательно, точка x^* принадлежит допустимой области. Градиент целевой функции в найденной точке равен нулевому вектору, а его норма равна 0, что дополнительно подтверждает стационарность найденного решения.

```
▶ # Итоговая проверка найденного решения

def objective_gradient(x):
    x1, x2 = x
    df_dx1 = 2 * x1 - 3 * (x2 - x1)**2
    df_dx2 = 3 * (x2 - x1)**2
    return np.array([df_dx1, df_dx2], dtype=float)

gradient_at_star = objective_gradient(x_star)
gradient_norm_at_star = np.linalg.norm(gradient_at_star)

final_check_data = {
    "x*": x_star,
    "f(x*)": objective_function(x_star),
    "g(x*)": constraints(x_star),
    "constraint_violation": constraint_violation(x_star),
    "feasible": is_feasible(x_star),
    "gradient": gradient_at_star,
    "gradient_norm": gradient_norm_at_star
}

for key, value in final_check_data.items():
    print(key, ":", value)
```

Рисунок 19 — Код итоговой проверки найденного решения

```

... x* : [0. 0.]
    f(x*) : 0.0
    g(x*) : [-1. 0.]
    constraint_violation : 0.0
    feasible : True
    gradient : [0. 0.]
    gradient_norm : 0.0

```

Рисунок 20 — Результат итоговой проверки найденного решения

3.12. Формирование итоговой таблицы результатов

После выполнения основных этапов решения была сформирована итоговая таблица результатов. В таблицу были включены номер варианта, координаты найденной точки условного минимума, значение целевой функции, значения ограничений, величина нарушения ограничений, норма градиента и признак допустимости решения.

Итоговая таблица показывает, что для варианта 2 найден условный минимум $x_{\min} = (0, 0)$. Значение целевой функции в этой точке равно $f(x_{\min}) = 0$. Оба ограничения выполняются: $g_1(x_{\min}) = -1$ и $g_2(x_{\min}) = 0$. Нарушение ограничений равно нулю, а норма градиента также равна нулю. Следовательно, найденная точка является допустимой стационарной точкой и может рассматриваться как точка условного минимума.

```

▶ # Формирование итоговой таблицы результатов

final_results = pd.DataFrame([
    {
        "Variant": variant,
        "x1*": x_star[0],
        "x2*": x_star[1],
        "f(x*)": objective_function(x_star),
        "g1(x*)": constraints(x_star)[0],
        "g2(x*)": constraints(x_star)[1],
        "Constraint violation": constraint_violation(x_star),
        "Gradient norm": gradient_norm_at_star,
        "Feasible": is_feasible(x_star)
    }
])

final_results

```

Рисунок 21 — Код формирования итоговой таблицы результатов

Variant	x1*	x2*	f(x*)	g1(x*)	g2(x*)	Constraint violation	Gradient norm	Feasible
0	2	0.0	0.0	-1.0	0.0	0.0	0.0	True

Рисунок 22 — Итоговая таблица результатов

4) Заключение

В ходе лабораторной работы были исследованы необходимые и достаточные условия существования условного экстремума функции с учетом ограничений. Для варианта 2 была рассмотрена задача поиска экстремума функции $f(x) = x_1^2 + (x_2 - x_1)^3$ при ограничениях $g_1(x) = x_1^2 + x_2^2 - 1 \leq 0$ и $g_2(x) = x_1 - x_2 \leq 0$.

В процессе выполнения работы была составлена функция Лагранжа, записаны и решены условия Куна–Таккера для различных случаев активности ограничений. После фильтрации допустимых решений для задачи условного минимума была найдена точка $x_{\min} = (0, 0)$, для которой значение целевой функции равно $f(x_{\min}) = 0$. Проверка ограничений показала, что $g_1(x_{\min}) = -1$ и $g_2(x_{\min}) = 0$, поэтому точка принадлежит допустимой области, а ограничение g_2 является активным.

Так как в задании требуется найти экстремум функции, дополнительно был выполнен анализ условного максимума. В результате среди допустимых точек Куна–Таккера была найдена точка $x_{\max} \approx (-0.784506, 0.620122)$, для которой значение целевой функции составляет $f(x_{\max}) \approx 3.386748$. В данной точке выполняются ограничения $g_1(x_{\max}) = 0$ и $g_2(x_{\max}) \approx -1.404627$, поэтому точка принадлежит допустимой области, а активным является ограничение g_1 .

Также была выполнена проверка условий второго порядка для найденной точки минимума. Матрица Гессе функции Лагранжа в точке $x_{\min}$ оказалась положительно полуопределенной, что подтверждает выполнение необходимого условия второго порядка

для локального минимума. Численная проверка методом SLSQP дала результат, совпадающий с аналитическим решением для минимума.

Таким образом, цель лабораторной работы была достигнута: были записаны и исследованы условия Куна–Таккера, выполнена проверка допустимости найденных точек, определены условный минимум и условный максимум функции для выбранного варианта. Условный минимум достигается в точке $x_{\min} = (0, 0)$, а условный максимум — в точке $x_{\max} \approx (-0.784506, 0.620122)$.